

NAME K.D.D. SaiAbhisramSUBJECT Deep learning lab work

STD. _____

DIV. _____

ROLL NO. _____

SCHOOL _____

Sr. No.	Page No.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Analyze and exploring the deep learning Platforms	24-07-23	He
2		Implement a classifier using open-source data set	7/8/23	He 11/11/23
3		study of the classifiers with respect to statistical parameters	7/8/23	He 11/11/23
4		Build a simple feed forward neural network to recognize hand written character	14/8/23	He 11/11/23
5		Study of activation function and its role	28-8-23	He
6		Implement gradient descent & back propagation in deep neural network	09-9-23	He 11/11/23
7		Build a CNN model to classify Cats & dogs Images	16-9-23	He 11/11/23
8		Experiment using LSTM		He 11/11/23
9		Build a RNN		
10		Perform Compression on MNIST data set using Autoencoder		He 11/11/23
11		to implement experiment using Variational Autoencoder		
12		Implement DCGAN to generate Colour images	27/10	He
13		understanding the architecture of Pre-trained model	27/10	He 11/11/23
14		Pre-trained CNN model as feature extractor using TLM		
15		Implement a Yolo model to detect objects		

Completed

14. Implement a Pre-trained CNN model as a feature extractor using transfer learning model

Aim :-

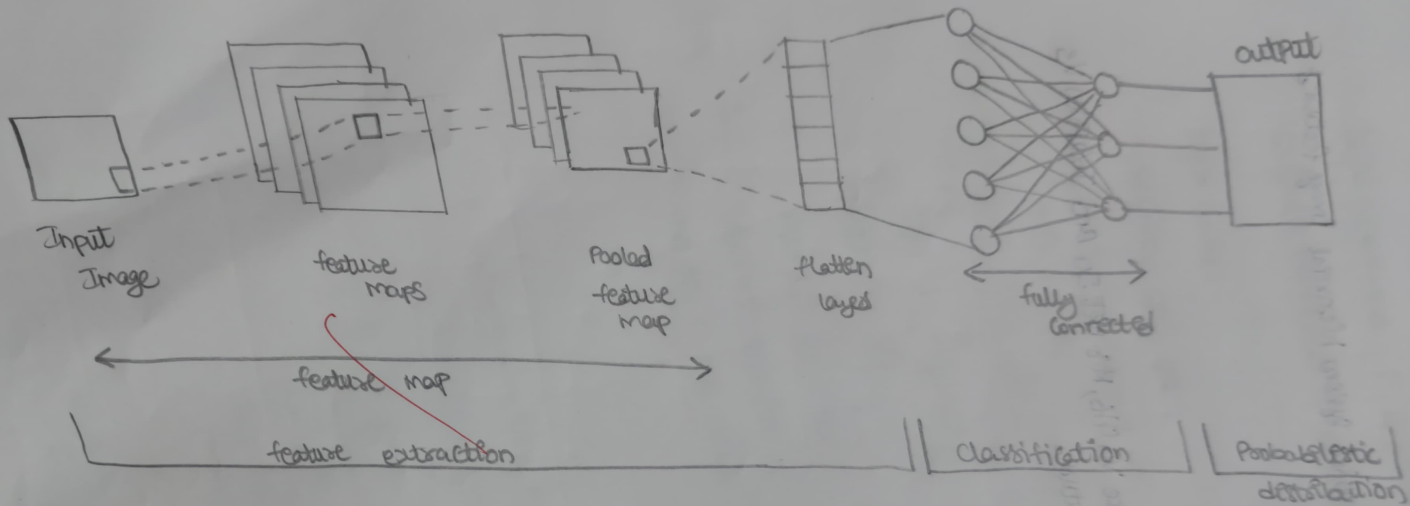
To implement a pre-trained CNN model as a feature extractor using transfer learning and use the extracted features of image.

Objective :-

- ① to understand the concept of transfer learning and feature extraction
- ② to use a pre-trained CNN model on trained Image net
- ③ to remove the top fully connected layers and use the convolutional base to extract features
- ④ to train a new classifier on the top of the extracted features of a new data set.
- ⑤ to visualize the model accuracy and loss during training

Pseudo Code

- ① Import required libraries
- ② Load a pre-trained CNN model
- ③ freeze convolutional layers to prevent training them
- ④ replace the top fully connected layers.
- ⑤ define loss function and optimizer
- ⑥ load dataset and create data loaders
- ⑦ train the classifier



CNN - pre trained with ReLU

[1]

✓ 14s



```
import torch
from torchvision import models

# Step 1: Load pre-trained ResNet-50
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
print("Model loaded successfully.")

# Step 2: Print full architecture
print(model)

# Step 3: Count total parameters
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total trainable parameters: {total_params}")

# Step 4: inspect specific layers
print("First convolutional layer:\n", model.conv1)
print("\nFinal fully connected layer:\n", model.fc)

# Step 5: torchsummary visualization
from torchsummary import summary
summary(model, (3, 224, 224))
```

```
from torchsummary import summary
summary(model, (3, 224, 224))
```



```
Linear(in_features=2048, out_features=1000, bias=True)
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 112, 112]	9,408
BatchNorm2d-2	[-1, 64, 112, 112]	128
ReLU-3	[-1, 64, 112, 112]	0
MaxPool2d-4	[-1, 64, 56, 56]	0
Conv2d-5	[-1, 64, 56, 56]	4,096
BatchNorm2d-6	[-1, 64, 56, 56]	128
ReLU-7	[-1, 64, 56, 56]	0
Conv2d-8	[-1, 64, 56, 56]	36,864
BatchNorm2d-9	[-1, 64, 56, 56]	128
ReLU-10	[-1, 64, 56, 56]	0
Conv2d-11	[-1, 256, 56, 56]	16,384
BatchNorm2d-12	[-1, 256, 56, 56]	512
Conv2d-13	[-1, 256, 56, 56]	16,384
BatchNorm2d-14	[-1, 256, 56, 56]	512
ReLU-15	[-1, 256, 56, 56]	0
Bottleneck-16	[-1, 256, 56, 56]	0
Conv2d-17	[-1, 64, 56, 56]	16,384
BatchNorm2d-18	[-1, 64, 56, 56]	128
ReLU-19	[-1, 64, 56, 56]	0
Conv2d-20	[-1, 64, 56, 56]	36,864
BatchNorm2d-21	[-1, 64, 56, 56]	128
ReLU-22	[-1, 64, 56, 56]	0
Conv2d-23	[-1, 256, 56, 56]	16,384
BatchNorm2d-24	[-1, 256, 56, 56]	512
ReLU-25	[-1, 256, 56, 56]	0
Bottleneck-26	[-1, 256, 56, 56]	0
Conv2d-27	[-1, 64, 56, 56]	16,384
BatchNorm2d-28	[-1, 64, 56, 56]	128
ReLU-29	[-1, 64, 56, 56]	0
Conv2d-30	[-1, 64, 56, 56]	36,864
BatchNorm2d-31	[-1, 64, 56, 56]	128
ReLU-32	[-1, 64, 56, 56]	0
Conv2d-33	[-1, 256, 56, 56]	16,384
BatchNorm2d-34	[-1, 256, 56, 56]	512
ReLU-35	[-1, 256, 56, 56]	0
Bottleneck-36	[-1, 256, 56, 56]	0

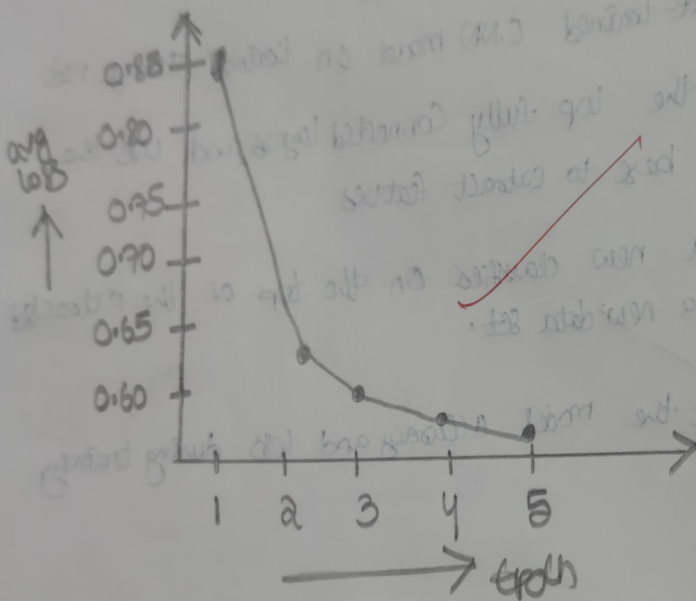


BatchNorm2d-44	[-1, 512, 28, 28]	1,024
Conv2d-45	[-1, 512, 28, 28]	131,072
BatchNorm2d-46	[-1, 512, 28, 28]	1,024
ReLU-47	[-1, 512, 28, 28]	0
Bottleneck-48	[-1, 512, 28, 28]	0
Conv2d-49	[-1, 128, 28, 28]	65,536
BatchNorm2d-50	[-1, 128, 28, 28]	256
ReLU-51	[-1, 128, 28, 28]	0
Conv2d-52	[-1, 128, 28, 28]	147,456
BatchNorm2d-53	[-1, 128, 28, 28]	256
ReLU-54	[-1, 128, 28, 28]	0
Conv2d-55	[-1, 512, 28, 28]	65,536
BatchNorm2d-56	[-1, 512, 28, 28]	1,024
ReLU-57	[-1, 512, 28, 28]	0
Bottleneck-58	[-1, 512, 28, 28]	0
Conv2d-59	[-1, 128, 28, 28]	65,536
BatchNorm2d-60	[-1, 128, 28, 28]	256
ReLU-61	[-1, 128, 28, 28]	0
Conv2d-62	[-1, 128, 28, 28]	147,456
BatchNorm2d-63	[-1, 128, 28, 28]	256
ReLU-64	[-1, 128, 28, 28]	0
Conv2d-65	[-1, 512, 28, 28]	65,536
BatchNorm2d-66	[-1, 512, 28, 28]	1,024
ReLU-67	[-1, 512, 28, 28]	0
Bottleneck-68	[-1, 512, 28, 28]	0
Conv2d-69	[-1, 128, 28, 28]	65,536
BatchNorm2d-70	[-1, 128, 28, 28]	256
ReLU-71	[-1, 128, 28, 28]	0
Conv2d-72	[-1, 128, 28, 28]	147,456
BatchNorm2d-73	[-1, 128, 28, 28]	256
ReLU-74	[-1, 128, 28, 28]	0
Conv2d-75	[-1, 512, 28, 28]	65,536
BatchNorm2d-76	[-1, 512, 28, 28]	1,024
ReLU-77	[-1, 512, 28, 28]	0
Bottleneck-78	[-1, 512, 28, 28]	0
Conv2d-79	[-1, 256, 28, 28]	131,072
BatchNorm2d-80	[-1, 256, 28, 28]	512
ReLU-81	[-1, 256, 28, 28]	0
Conv2d-82	[-1, 256, 14, 14]	589,824

Output

Epoch [1/5] ; loss : 0.8481
Epoch [2/5] ; loss : 0.6266
Epoch [3/5] ; loss : 0.5937
Epoch [4/5] ; loss : 0.5797
Epoch [5/5] ; loss : 0.5699

Training loss curve - transfer learning Parameters



Conclusion

1. Transfer learning is a powerful technique for improving model performance.
2. It allows us to leverage knowledge from pre-trained models.
3. This is particularly useful when we have limited data for the target task.
4. By sharing information across tasks, we can achieve better results.
5. Transfer learning is a key concept in deep learning and AI.
6. It is a technique that allows us to reuse knowledge from one task to another.

- ⑧ compute accuracy and loss per epoch
- ⑨ visualize training and validation accuracy
- ⑩ test the model and display performance

Observation

- * training time greatly reduced compared to training from scratch.
- * the model achieves high accuracy with limited epochs.

Result

Pre-trained model of CNN successfully implemented

~~exp~~